

Speaker script — EAI SecureComm 2026

Localizing Hot Pixels in Android Malware Images R. Can Oztas · Sevil Sen — WISE Lab, Hacettepe University

Target: **15:00** talk + Q&A. ~1,850 words at a calm ~125 wpm. Timings are cumulative clock marks, not durations.

0:00 — Title (PDF p.1 · title)

Good morning. I'm Can Oztas, from the WISE Lab at Hacettepe University, and this is joint work with Sevil Sen.

Our paper is about what happens *after* an Android malware detector says “yes.” We take image-based detection and push it down to the specific Smali classes an analyst can actually open.

0:20 — Motivation (PDF p.3 · footer 1/18)

The volume is the easy part of the story. In 2024 alone, Kaspersky blocked 33.3 million attacks on Android and saw 1.1 million new malicious installers.

But detection is largely a solved problem. If you hand me an APK, a good classifier will tell you it's malicious with high accuracy. That's not the question an analyst is stuck on. The question they're stuck on is: **which class inside this APK is malicious, and why?**

And this is where the image-based line of work stops short. Methods like DexRay treat the DEX bytes as pixels and classify the whole APK — one verdict for the entire file. Methods that add explainability, like ImageDroid with Grad-CAM, highlight a *region* of the image. But a bright region on a greyscale image is not something you can open in a disassembler. Nobody has resolved it back to code.

So our proposal — Hot Pixel Localization — is a CNN plus an XAI ensemble, plus, critically, an **invertible** pixel-to-byte-to-Smali mapping.

1:20 — Contributions (PDF p.4 · footer 2/18)

Four contributions, quickly.

One: a fully static pipeline that unifies image-based detection with class-level localization. Two: evaluation on both detection and localization, using real *and* synthetic malware. Three: an auditable mapping — every heatmap coordinate traces back to a DEX byte range and then to a Smali class. Four: the implementation is public.

2:05 — Related work (PDF p.5 · footer 3/18)

This table is the one-slide version of where we sit. Look at the last column.

Image-based works stop at region-level explanations. Sequence and graph based works — Droidetec, MKLDroid, XAIDroid — do reach class or method granularity, but they give up the two-dimensional structural view of the binary.

HPL is the first to bridge image XAI to Smali-level artifacts. That's the gap.

(If running long: read only the last column and move on — 20 seconds.)

2:25 — Pipeline (PDF p.7 · footer 4/18)

Here's the whole pipeline on one slide. APK, concatenate the DEX, make an image, classify it with a CNN, produce a heatmap.

Everything up to here exists in the literature. **The dashed red box is our contribution:** the inverse map that takes a hot pixel back to bytes, and the ranked list of Smali classes that comes out the other end.

Detection gives you the signal. The inverse map is what turns that signal into something an analyst can open.

3:40 — Step 1, DEX to image (PDF p.8 · footer 5/18)

Step one. We concatenate all `classes*.dex` into one byte stream of length L , and reshape it into a square of side S — the ceiling of the square root of L — zero-padded. Then we resize that to a fixed 256×256 for the CNN.

The important detail is the last bullet. For every sample we persist the triple: original side S , network side S -nought, and true length L . That triple is what makes the transform **invertible**. Without it you have a picture of a binary; with it, you have a coordinate system you can walk backwards.

Note the square is sample-specific — it's not a fixed grid. That preserves traceable packing.

4:40 — Step 2, the CNN (PDF p.10 · footer 7/18)

The classifier itself is deliberately unremarkable — a VGG-style network: three convolutional blocks, 32 to 64 to 128 filters, then a dense layer and a sigmoid. Trained on KronoDroid: about 41,000 malware and 37,000 benign, five-fold cross-validation.

One design choice worth defending: we train it **binary**, not multi-class by family. Analyst triage cares about malicious *regions*, not family labels, and attribution learned this way transfers across families.

For localization we evaluate on 105 real samples from XAIDroid — seven families, fifteen each — and on synthetic Mystique samples, where payloads are injected into benign carriers so ground truth is exact.

5:25 — Step 3, the attribution ensemble (PDF p.11 · footer 8/18)

Step three. We don't trust a single explainer, and neither should you.

We run five: two from the CAM family — Grad-CAM++ and Score-CAM — which are coarse and structural; and three gradient methods — vanilla saliency, SmoothGrad, and Integrated Gradients — which are fine and pixel-level.

We standardize all five maps, then take a **pixel-wise maximum** across them. That's Ensemble Max.

Why max? Because the two families fail differently. CAM anchors structural blobs; gradients sharpen edges. Integrated Gradients in particular struggles on discrete bytes — path integration assumes smooth natural images. Taking the max means one method's failure mode doesn't sink the result. And the output isn't just a pretty map: it feeds a consolidated per-class score list.

6:40 — Step 4, pixel to class (PDF p.12 · footer 9/18)

This is the core of the paper, so let me be precise.

Given a hot pixel at coordinates x, y on the resized map, we invert the resize to get back to the original S -by- S grid, then flatten: index equals y times S plus x . That gives a byte offset in the original, unpadded DEX stream.

But resizing is lossy — one pixel on the 256-grid covers an *interval* of bytes, not a single byte. So rather than interpolating, we take the maximum saliency across that whole support interval. We call this support-region max-spreading.

Then baksmali disassembles, androguard parses the DEX structure, and we resolve the byte range to the Smali class that contains it — filtered against a whitelist of benign libraries like AndroidX and okhttp.

And we sanity-checked the round trip on all 105 samples. Class-match hit rate: **1.00**. Positional error, median and 95th percentile: **zero bytes**. The support region is 261 bytes wide at the median — which is small next to a typical Smali class, so it resolves cleanly. The mapping is exact, not approximate.

8:10 — Detection results (PDF p.14 · footer 10/18)

Detection first, briefly. Precision 96.2, recall 96.3, false positive rate one percent.

I want to be clear about what this slide is *not*. Detection is not our contribution. This number exists to establish that the DEX-image representation is competitive at scale — that it's a solid **substrate** for the attribution methods to interpret. If detection were weak, localization on top of it would be meaningless.

8:55 — Localization results (PDF p.15 · footer 11/18)

This is the result that matters. AUC and NSS, on real malware and on synthetic.

Ensemble Max: **AUC 0.89, NSS 1.71** on real malware. It's the most consistent method across both real and synthetic — no individual explainer wins on both.

Notice Integrated Gradients on its own: 0.55 AUC, barely better than chance. That's exactly the failure mode I mentioned, and exactly why the ensemble matters — the max absorbs it.

One honest caveat: localization quality tracks payload contiguity. Families whose malicious code sits in one contiguous block — apofer, opfake — localize cleanest. Scattered payloads are harder.

10:10 — From hot pixel to class (PDF p.17 · footer 12/18)

Before the case studies, the framing. There are two different questions here.

The evaluation question — does the heatmap land on ground-truth malicious classes? — is what AUC and NSS answer. But the analyst question is different: **is the top-ranked class actually useful when you open it?**

So we manually reverse-engineered the top hits for three families. Three different behaviours: SMS fraud, an obfuscated overlay, and a dropper.

10:55 — Case 1, agent.aax (PDF p.18 · footer 13/18)

Agent.aax sends SMS to premium-rate numbers without consent.

Our top-ranked class was `com/software/app/R` — the resource class. And here's the thing: **it was not in the ground-truth set**. By the metric, that's a miss.

But open it. It holds `error_sms_sending`, `megafon_rules` — a Russian telecom operator — and `register_path`, the C2 registration endpoint. Plus a pointer to the raw resource holding the premium numbers and SMS templates.

That's the campaign's entire configuration, in one class the label set never contained. The ground truth listed executable classes only — no executable code, so nobody labelled it. Non-executable is not the same as uninteresting. HPL reads *bytes*, so it sees configuration that a code-centric ground truth cannot.

Opfake.bo makes a related point: every package and class name is replaced with random strings, and our top hit was still the configuration and constants class — timing constants and decryption-initialised strings. Renaming defeats signatures; it doesn't defeat byte-level structure.

(Case 2 is folded into one sentence here to protect the clock. If you have time, its own slide is next.)

12:10 — Case 3, wapnor.a (PDF p.20 · footer 15/18)

The dropper. And this is my favourite result.

Rank one is `SROBWZb` — an exact match with the ground-truth payload class. Good, but expected.

Rank two is the interesting one: `aqrPxuqGxP`. Here it is — the whole class, straight from `baksmali`.

It extends `android.app.Application`, and it overrides `attachBaseContext`. Three instructions: call super, then hand two obfuscated strings to `SROBWZb` — the class ranked number one.

`attachBaseContext` fires at the *earliest* point of the app lifecycle: before `onCreate`, before any Activity exists, and before most dynamic-analysis instrumentation has attached.

So the ranking isn't just a list. It's a narrative: the payload at rank one, and its ignition mechanism at rank two.

13:25 — Obfuscation (PDF p.22 · footer 16/18)

Does this survive obfuscation? We applied Obfuscapk with three techniques targeting different layers: arithmetic branches for control flow, advanced reflection for API calls, and constant-string encryption for data.

Ensemble Max holds at **AUC 0.856**, **NSS 1.55** — down, but not broken.

The effect is family-dependent. Some transformations genuinely disrupt learned structural patterns and introduce new salient regions. But localization stays usable overall, because the ensemble absorbs individual explainer failures.

14:25 — Take-away (PDF p.24 · footer 18/18)

To close.

HPL turns image-based CNN detection into class-level, analyst-inspectable localization. The key ingredient is the invertible transform plus support-region back-projection — every heatmap coordinate is auditable.

Ninety-six percent detection, AUC 0.89 and NSS 1.71 for localization.

But the case studies are the real argument. HPL surfaced a configuration class the ground truth omitted, held up under full identifier randomisation, and recovered a dropper's ignition mechanism — not just its payload.

Limitations, honestly: there is no public pixel-level mask for DEX images, so we derive ground truth from class annotations. We're DEX-only — native code and heavy dynamic loading are invisible to us. And like any learned detector, this needs periodic recalibration against drift.

Code is at the link. Thank you — happy to take questions.

15:00 — Questions (PDF p.25 · footer 18/18)

Delivery notes

If you're running long, cut in this order:

1. Slide 22 (obfuscation) → “it holds up under Obfuscapk, details in the paper”
2. Slide 14 (detection) → just say the numbers, skip the framing
3. Slide 11 → compress to “five explainers, pixel-wise max, because they fail differently”

Do **not** cut slide 12 (the inverse mapping) or the case studies. That is the paper's actual claim.

Q&A preparation

Ordered roughly by how likely they are. **Bold** is the one-sentence answer — say that first, then expand only if they push.

On the method

1. Why pixel-wise max and not mean, or a learned weighting?

Because the five explainers fail differently, and averaging lets a failure drag down a correct signal. Integrated Gradients gets AUC 0.55 on its own — essentially chance. In a mean, that noise is a quarter of every pixel's value. Max keeps the strongest evidence at each pixel, so a method that is confidently right somewhere still wins there, and a method that is uniformly weak never hurts. We didn't try a learned weighting because it would need a pixel-level-labelled training set, which is exactly what doesn't exist here.

2. Why is the transform invertible? Isn't resizing lossy?

The resize is lossy; that's why we don't invert it pointwise. We keep the triple (S, S_0, L) per sample, so we can always get from a 256-grid coordinate back to a *range* on the original $S \times S$ grid, and from there to a byte interval. Rather than interpolating to a single byte, we take the max saliency across the whole support interval. The median interval is 261 bytes, which is small relative to a Smali class — so it resolves to the right class. Round-trip class-match hit rate is 1.00 on 105 samples.

3. Doesn't zero-padding the tail create artifacts the CNN learns?

Padding is a constant region with no gradient structure, and it sits at a sample-specific position, so it can't act as a stable shortcut. We also exclude padded offsets when back-projecting, since they map beyond L .

4. Why a square? Why not a fixed-width row layout like Nataraj?

The square is what keeps the packing traceable. A fixed width means the row length is unrelated to the file, so the 2-D neighbourhood is arbitrary. Sample-specific $S = \lceil \sqrt{L} \rceil$ makes the mapping a pure function of L , which is what we persist and invert.

5. Why binary classification rather than family multi-class?

Because triage cares about malicious regions, not family names. Attribution learned from a binary objective transfers across families; a family head would tie the explanation to labels that are noisy and inconsistent across vendors.

On the evaluation

6. Your ground truth is class-level, but you claim pixel-level localization. Isn't the metric mismatched?

Yes, and we say so — this is our first stated limitation. There is no public pixel-level mask for DEX images. We derive a pixel-level ground truth by marking the byte ranges of classes annotated as malicious. That's why we also use Mystique, where payloads are *injected*, so the byte ranges are known exactly rather than inferred. The two agree, which is the strongest check available to us.

7. AUC 0.89 sounds high — what's the baseline? Isn't malicious code a large fraction of these files?

NSS is the answer to that. AUC can flatter you when the positive region is large; NSS normalises against the map's own distribution. We report NSS 1.71, meaning saliency at true malicious bytes sits 1.7 standard deviations above the map mean. Random attribution gives NSS 0.

8. Only 105 real samples and 7 families. Isn't that thin?

It is thin, and it's bounded by the annotation cost — every sample was manually verified. Detection is evaluated at scale (78k samples on KronoDroid); localization is the part that needs hand-labelled ground truth. We report per-family numbers rather than a pooled figure precisely so you can see the variance across families.

9. Case 1 counts as a miss by your own metric, yet you present it as a win. Isn't that having it both ways?

It is a miss by the metric, and I'd rather show it than hide it. The point is what the miss reveals: the ground truth lists executable classes only, so a resource class holding the premium-SMS templates and the C2 registration path was never labelled. Our AUC is therefore *penalised* for a genuinely useful hit. That's an argument that class-level annotation is the weak link, not the method.

On robustness and threat model

10. An adversary who knows HPL — how would they defeat it?

Move the malicious logic out of the DEX, or break the contiguity the CNN keys on. Native code and dynamic loading are already outside our scope. Within DEX, deliberately fragmenting a payload across many small classes would spread saliency and lower NSS. We don't claim adversarial robustness — Obfuscapk tests *common* evasion, not targeted attacks against our pipeline.

11. Obfuscation dropped AUC to 0.856 — for wapnor it fell 0.21. Isn't that fragile?

It's family-dependent, and the direction is informative. Some transformations — constant-string encryption especially — genuinely relocate and reshape the byte regions the model learned. Two families actually *improved*, because the transformation made the payload more distinctive. Median behaviour is a modest degradation, not a collapse.

12. Packers? App bundles with multiple DEX files? Split APKs?

Multi-DEX is handled — we concatenate `classes*.dex` in order and the offset arithmetic accounts for it. Packers that decrypt at runtime are out of scope; HPL is

fully static by design, and we state that as a limitation.

On positioning

13. How is this different from XAIDroid, which already reaches class level?

They start from an API-call graph; we start from the raw byte image. That means no graph extraction step and no dependence on API-resolution succeeding under obfuscation — but it also means we keep the 2-D structural view, which is what makes the visual explanation auditable back to byte offsets.

14. Why should an analyst trust a heatmap at all?

They shouldn't have to — that's the point of the inverse map. The output isn't a picture, it's a ranked list of Smali classes with byte ranges. The analyst opens the class and judges for themselves. The heatmap is the mechanism, not the deliverable.

15. Is the code and data available?

Code is public at the link on the take-away slide. The KronoDroid, XAIDroid, and Mystique datasets are third-party and available from their original authors.

If you don't know

Say so, then offer the nearest thing you do know:

“We didn't test that — what we did test is X, and I'd expect Y, but I don't want to guess on the record. Find me afterwards and I'll follow up.”

Do **not** speculate about numbers you didn't measure.